浙江大学

人工智能

大 程 序 报 告

1. 姓名 ： 林炬乙

2. 姓名 ： 蒋宇龙

3. 姓名： 蒋理

指导老师： 龚小谨

2020~2021 春夏学期 ２０２１ 年 ６ 月 16 日

目 录

1 大程序简介 3

2 程序开发设计 4

2.1 实现流程简述 4

2.2 算法设计 8

2.2.1 构建 agent 8

2.2.2 获得当前状态 9

2.2.3 得分区域的划分: 10

2.2.4 更新 q_table 函数 10

2.2.5 get_best_action 函数 11

2.2.6 reward 函数构造 11

2.2.7 优化与参数调整 12

2.3 遇到的问题以及解决方式 13

2.3.1 绘图问题 13

2.3.2 如何在有限时间内评估 13

2.3.3 接口问题 14

2.3.4 游戏运行速度较慢 14

2.3.5 收敛的位置不正确 14

2.3.6 游戏运行时，会带有拖影效果 14

3 团队合作 15

3.1 任务分工 15

3.2 编码规范 15

3.3 个人收获 17

林炬乙收获感言: 17

蒋理收获感言: 17

蒋宇龙收获感言： 17

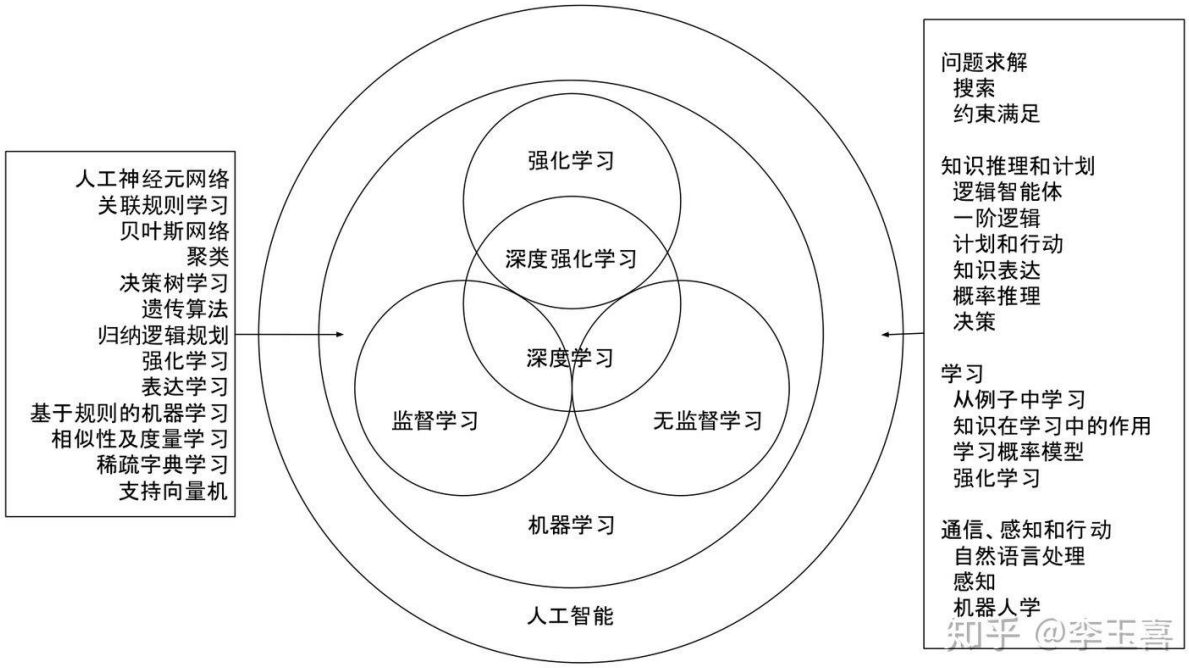
Flappy Bird 大程序设计

1 大程序简介

基于本学期对人工智能的学习，使用 python 进行完整的程序设计，针对程序设计题目与要求，分析功能要求、划分功能模块.

应用预期目的:本次程序设计的主题是 Flappy Bird，游戏中玩家必须控制一只小鸟，跨越由各种不同长度水管所组成的障碍。在《Flappy Bird》这款游戏中，玩家只需要**用一根手指来操控**，**点击触摸屏幕**，小鸟就会**往上飞**，不断的点击就会不断的往高处飞。放松手指，则会快速下降。所以我们的预期目标就是 agent 要控制小鸟一直向前飞行.

如何选择合适方法解决该问题:



对于传统的机器学习方法而言，最有效的问题是涉及结构化数据的数据，即结构和标签之间的关系已经被理解的数据。例如，一个数据表将一个人的某些特征（例如年龄，孩子的数量，吸烟者或不吸烟者，已知的健康状况等）与该人的健康保险的价格相匹配。

强化学习就是一种机器学习的方法，强化学习智能体（agent）与环境（environment）交互，针对序列决策问题，通过试错（trial-and-error）的方式学习最优策略。强化学习一般定义为马尔科夫决策过程(Markov Decision Process, MDP). 在每一个时间步骤，智能体接受到一个状态（state)，根据策略（policy）选择一个动作（action)，获得奖赏（reward)，然后根据环境的动态模型转移到下一个状态。这里面，策略表达智能体的行为，就是状态到动作的映射。强化学习中，经验（experience）是指（状态，动作，奖赏，下一个状态）这样一系列的数据。用一个折扣因子(discount factor)来表达将来的奖赏对现在的影响。模型（model）指状态转移模型和奖赏函数。强化学习的适用范围非常广泛：状态和动作空间可以是离散的或连续的。我们可以阐述一下为什么不用神经网络:

由于存在一些问题，特征和目标之间的关系不太清楚，而神经网络将是对数据进行预测的最佳选择。这些将是涉及非结构化数据（例如图像，音频和自然语言文本）的问题。例如，猫的图像（特征）中的像素排列方式使其更像猫（标签）的图片，而不是其他任何东西。

我们这个项目不需要用神经网络，因为特征和目标关系还是非常清楚的.

我们采用最经典的 Q learning 算法，它是一种价值导向，价值迭代的强化学习算法. QLearning 是强化学习算法中 value-based 的算法，Q 即为 Q（s,a）就是在某一时刻的 s 状态下(s∈S)，采取 动作 a (a∈A)动作能够获得收益的期望，环境会根据 agent 的动作反馈相应的回报 reward r，所以算法的主要思想就是将 State 与 Action 构建成一张 Q-table 来存储 Q 值，然后根据 Q 值来选取能够获得最大的收益的动作。

Q-learning 存在的问题：

（1）Q-learning 需要一个 Q table，在状态很多的情况下，Q table 会很大，查找和存储都需要消耗大量的时间和空间。

（2）Q-learning 存在过高估计的问题。因为 Q-learning 在更新 Q 函数的时候使用的是下一时刻最优值对应的 action，这样就会导致“过高”的估计采样过的 action，而对于没有采样到的 action，便不会被选择为最优的 action。

2 程序开发设计

2.1 游戏实现

flappy bird 的游戏部分是基于 python 的 pygame 游戏库实现的，其中贴图素材来源于网络的 flappy bird 素材包。在游戏中，一个界面共有三个元素：小鸟、管道、背景界面。其中小鸟和管道都通过 python 的类对象来实现，背景界面则通过定期刷新贴图的方式实现更新。

2.1.1 整体游戏逻辑

1. 创建游戏界面，初始化小鸟和管道对象
2. 启动游戏时钟控制帧率
3. 根据当前所在状态（开始界面、游戏界面、结束界面），对每一帧进行背景界面的绘制
4. 在游戏中检测输入按键，对小鸟和管道的状态实时更新，并绘制实时状态
5. 对当前各项数值做判断，实现状态跳转
6. 更新画布、进入下一帧

其具体代码如下图:

```python
# 初始化
screen = init_game()
bird, pipe_sprites = init_sprite()
clock = pygame.time.Clock()

while True:
    # 开启时钟
    clock.tick(60)
    # 如果是菜单界面
    if is_start == False:
        DrawBackground(screen)
        DrawStartMenu(screen)
        bird.draw(screen)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN:
                is_start = True
    # 如果是游戏界面
    else:
        restart = press(is_game_running, bird)
        if restart:
            return
        # 画界面
        DrawBackground(screen)
        DrawScore(screen,score,score_img)
        # 判断输入事件
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.MOUSEBUTTONDOWN or event.type == pygame.KEYDOWN:
                bird.up()
        # 判断游戏是否结束
        is_collision = collision(bird, pipe_sprites)

        # 画管道
        for pipe in pipe_sprites:
            pipe.draw(screen)

        bird.draw(screen)  # 画鸟

        if is_collision:
            is_game_running = False  # 如果碰撞 游戏结束
            best = max(best,score)
        if is_game_running:
            is_add_pipe, score = move_pipe(bird, pipe_sprites, is_add_pipe, score)  # 不碰撞 移动管道
        else:
            Gameover(screen,score,best,result_img)
    # 更新画布
    pygame.display.update()
```

2.1.2 主函数中的函数和类实现

对于 pygame 的绘图，该库提供了两种方式，一是直接调用画图工具手动绘制，二是在指定位置放置贴图。此处采用贴图的方式，即通过 image.load 读取本地图片，通过 screen.blit 在指定的位置显示图片。在 pygame 库中，所有的图片显示都是以左端、上端为基点的，因此在后面所有的位置定义中都需要定义对象的左边界和上边界。部分绘图函数如下：

```
day = pygame.image.load('./sources/background/day.png')
land = pygame.image.load('./sources/menu/land.png')
title = pygame.image.load('./sources/menu/title.png')
tutorial = pygame.image.load('./sources/menu/tutorial.png')
text_game_over = pygame.image.load('./sources/gameover/text_game_over.png')
score_panel = pygame.image.load('./sources/gameover/score_panel.png')
score_img = [pygame.image.load('./sources/menu/menu_score_0.png'),pygame.image.load('./sources/menu/
             pygame.image.load('./sources/menu/menu_score_3.png'),pygame.image.load('./sources/menu/
             pygame.image.load('./sources/menu/menu_score_6.png'),pygame.image.load('./sources/menu/
             pygame.image.load('./sources/menu/menu_score_9.png')]
result_img = [pygame.image.load('./sources/gameover/result_score_0.png'),pygame.image.load('./source
              pygame.image.load('./sources/gameover/result_score_3.png'),pygame.image.load('./source
              pygame.image.load('./sources/gameover/result_score_6.png'),pygame.image.load('./source
              pygame.image.load('./sources/gameover/result_score_9.png')]
gold = pygame.image.load('./sources/gameover/medals_1.png')
silver = pygame.image.load('./sources/gameover/medals_2.png')
corper = pygame.image.load('./sources/gameover/medals_3.png')
```

```
def DrawBackground(screen):
    # 显示背景图和地面
    screen.blit(day,(0,0))
    screen.blit(land,(0,BASE_HEIGHT))


def DrawStartMenu(screen):
    # 显示标题
    screen.blit(title, ((SCREEN_WIDTH - 178)/2, 80))
    # 显示引导按钮
    screen.blit(tutorial,(SCREEN_WIDTH * 0.2 - 33,(BASE_HEIGHT - BIRD_HEIGHT)/2 + BIRD_HEIGHT + 1))

def DrawScore(screen,score,score_img):
    left = SCREEN_WIDTH/2 -12
    if(score == 0):
        screen.blit(score_img[score],(left , BASE_HEIGHT/4))
    while(score != 0):
        score = int(score)
        temp = score%10
        screen.blit(score_img[temp],(left , BASE_HEIGHT/4))
        left = left - 24
        score = int(score/10)
```

对于小鸟类的实现，我们需要定义其为 pygame 中精灵类别的子类，用于继承精灵类的函数。类内需要定义以下参数：小鸟的当前位置（pygame.rect），随重力下降的速度，每次跳跃的高度，死亡状态。在主函数中，每一次鼠标点击都会触发一次小鸟跳跃，即 bird.up 函数，用于将小鸟的跳跃状态置一，如果已经在跳跃状态，则初始化跳跃速度。

小鸟实际位置的更新则是在 update 函数中完成的，其逻辑是随着时间流逝，跳跃上升速度会变慢，而下降速度会越来越快。根据上升和下降的速度，可以更新 bird.rect 来修改 bird 的实时位置。为了防止

鸟出现在规定边界外面的 bug，我们需要使鸟碰到上下边界后，速度强制置 0，并游戏结束。

最后，还需要一个 draw 函数来控制小鸟这个对象的贴图更新。

对于管道类的实现，我们同样定义为精灵类别的子类。每个管道的参数为管道位置（以左端和上端定义），管道高度，计分状态。其中，计分状态用于解决鸟飞过管道后每一帧都会重复计分的情况，若计分状态置 1，则该管道计过分，接下来不计分。

在初始化时，我们需要根据界面上下边界的高度来生成随机长度的上下管道，且保证管道间距离大于某一阈值，以便鸟能够飞过去。且为了游戏难度考虑，同一帧内必须存在两组管道，即 1 上、1 下、2 上、2 下四个。这可以通过精灵类下的 Group 类来定义，其作用类似一个列表，表内存放的是精灵类对象。完成管道类的生成后，通过同样的 draw 函数进行管道的绘制。

在每一帧中，我们还需要进行管道的移动（move_pipe）和碰撞的检测（collision），并以此进行状态切换的判断。两个管子之间的间隙等于一个固定 gap150 加上一个随机值,管道移动即简单地把管道的位置向左移动，如果超出地图边界，则可以删除该管道，并在 group 中添加新的管道。完成移动后，可以同时进行加分的检测，即鸟飞过管道中点完成加分。

碰撞检测调用了精灵类自带的函数 collide_rect，检测两个对象的 rect 属性是否有区域交叉，并返回一个布尔值。除了鸟和管道的碰撞外，还要检测上下边缘的碰撞。最终将碰撞状态返回到主函数中进行状态切换。

至于结束界面绘制、分数统计等，均为重复性工作和简单的位置调整，因此不赘述。游戏的总体参数如下：

```
FPS = 60
# 屏幕大小
SCREEN_WIDTH = 288
SCREEN_HEIGHT = 512
# 鸟大小
BIRD_WIDTH = 45
BIRD_HEIGHT = 45
# 管道大小
PIPE_WIDTH = 50
PIPE_HEIGHT = 300
# 管道之间间隔
PIPE_GAP = 150
# 管道移动速度
PIPE_SPEED = 4
# 地面高度
FLOOR_HEIGHT = 96
# 游戏有效高度
BASE_HEIGHT = SCREEN_HEIGHT - FLOOR_HEIGHT
```

2.2 算法设计

我们采用 Q-learning 算法，构建并维护一个 Qtable，设定了一些列状态来划分参数、设置了独特的 reward 函数用于加快收敛速度，在反复迭代多次后该表将会收敛到一个稳定值，为小鸟提供了最优的决策方案。

2.2.1 构建 agent

```
class Agent():
    def __init__(self):
        #agent 的 q-table,
        self.qtable = np.zeros((8,8,8,2))
        #学习率
        self.alpha = 0.4
        #奖励衰减因子
        self.gama = 0.8
        #贪婪率
        self.greedy = 0.8
```

该 agent 初始化时会自动生成一张 qtable，本质为一个 8*8*8*2 的四维矩阵，前三维分别是小鸟垂直、水平距离、速度的级别划分，最后一维是动作，即是否跳跃。学习率 alpha 是历史数据比例. alpha 越低，历史数据占比就高，小鸟会更多地凭借历史经验来采取决策，学习较慢。 如果 alpha 较高则遗忘速度变快，

我们根据一般经验选择 alpha = 0.7。gamma 是奖励衰减因子，greedy 是贪婪因子，目的是防止陷入局部最优解，二者可以作为优化选项，并不是算法所必须的内容

2.2.2 获得当前状态

Get_state 函数

通过这个函数得到当前状态，返回值三个值分别为：

state[0]：当前距离上方管道底部的距离级别

state[1]：距离下一管道的水平距离级别

state[2]：小鸟当前的跳跃速度级别

```python
def get_state(pipe_sprites,Bird):
    """
    得到当前状态，三个值分别为:
    state[0]: 距离上方管道底部有多远
    state[1]: 距离管道的水平距离为多远
    state[2]: 小鸟当前的跳跃速度
    """
    return_state = np.zeros((3,), dtype=int)
    dist_to_pipe_horz = 0
    dist_to_pipe_bottom = 0
    for pipe in pipe_sprites:
        dist_to_pipe_horz = pipe.rect.right - 20 - Bird.rect.left
        dist_to_pipe_bottom = Bird.rect.top - (pipe.rect.top + 320)
        if dist_to_pipe_horz <= 0:
            continue
        break
```

```python
    if velocity < 2:          #very slow
        velocity_category = 0
    elif velocity < 4:
        velocity_category = 1
    elif velocity < 6:
        velocity_category = 2
    elif velocity < 8:
        velocity_category = 3
    elif velocity < 10:
        velocity_category = 4
    elif velocity < 12:
        velocity_category = 5
    else:
        velocity_category = 6 #fast
```

```python
#高度的阈值
if dist_to_pipe_bottom < -40:  # very close or less than 0
    height_category = 0
elif dist_to_pipe_bottom < 0:
    height_category = 1
elif dist_to_pipe_bottom < 40:  # close
    height_category = 2
elif dist_to_pipe_bottom < 75:  # not close
    height_category = 3
elif dist_to_pipe_bottom < 110:  # mid
    height_category = 4
elif dist_to_pipe_bottom < 150:  # far
    height_category = 5
elif dist_to_pipe_bottom < 190:  # very far
    height_category = 6
else:
    height_category = 7
```

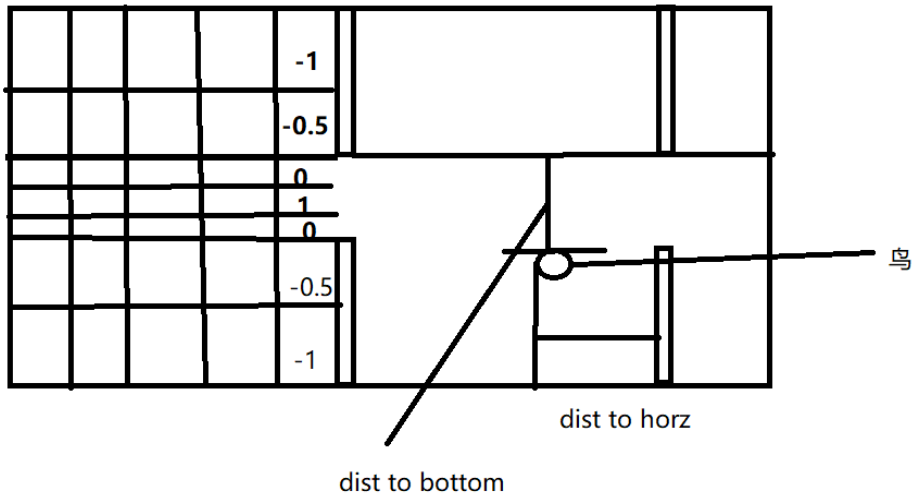
```python
if dist_to_pipe_horz < 40:  # very close
    dist_category = 0
elif dist_to_pipe_horz < 60:  # close
    dist_category = 1
elif dist_to_pipe_horz < 80:  # not close
    dist_category = 2
elif dist_to_pipe_horz < 120:  # mid
    dist_category = 3
elif dist_to_pipe_horz < 160:  # far
    dist_category = 4
else:
    dist_category = 5

return_state[0] = height_category
return_state[1] = dist_category
return_state[2] = velocity_category
return return_state
```

该函数获得小鸟的目前位置状态信息，针对不同的位置做出不同的决策，通过 state 来查询 qtable 从而选择是否跳跃，多细分阈值有助于优化 agent. 状态数目越多，获得的信息也越多，但付出的代价是

所需的训练迭代次数也变多，平均来讲，完全收敛需要几万场游戏。

2.2.3 得分区域的划分:



这是一张小组成员手绘的草图，用于展现区域的划分细节，与分工时的沟通方便，左边是理想的区域划分,右边是各个距离参数的物理意义，管子的长度为 320，游戏中只定义了管道 top 位置，因为容易得到管子下端位置为 320+top

2.2.4 更新 q_table 函数

Q-learning 的核心思想在于 bellman 方程，在本例中，其形式如下：

$$Q^{new}(s_t,a_t) \leftarrow \underbrace{Q(s_t,a_t)}_{\text{旧的值}} + \underbrace{\alpha}_{\text{学习率}} \cdot \overbrace{\underbrace{\left(\underbrace{r_t}_{\text{奖励}} + \underbrace{\gamma}_{\text{奖励衰减因子}} \cdot \underbrace{\max_a Q(s_{t+1},a)}_{\text{estimate of optimal future value}}\right)}_{\text{new value (temporal difference target)}} - \underbrace{Q(s_t,a_t)}_{\text{旧的值}}}^{\text{temporal difference}}$$

```python
def update_q_table(q_table, old_state, current_action, next_state, r,alpha):
    """
    根据一次动作前后的两个状态更新Q-table
    """
    next_max_value = np.max(q_table[next_state[0], next_state[1], next_state[2]])

    q_table[old_state[0], old_state[1], old_state[2],current_action] = (1 - alpha)\
        * q_table[old_state[0], old_state[1], old_state[2], current_action]\
        + alpha * (r + next_max_value)
    return q_table
```

1-alpha 是以前的数据比例. alpha 低了,以前数据比例就高，所以故步自封不会学习新知识.

2.2.5 get_best_action 函数

根据当前状态来查询 Qtable，获得 q 值，根据 q 值来选择 jump 或 no jump, 返回最佳的策略。

```python
def get_best_action(q_table, state, greedy=False):
    # 获得q值,通过state和qtable来选择是跳跃还是不跳跃
    jump = q_table[state[0], state[1], state[2], 0]
    no_jump = q_table[state[0], state[1], state[2], 1]
    # 是否执行贪婪策略（默认否）
    if greedy:
        if np.random.rand(1) < greedy:
            return np.random.choice([0, 1])
        else:
            if jump > no_jump:
                return 1
            else:
                return 0
    else:
        if jump > no_jump:
            return 1
        else:
            return 0
```

2.2.6 reward 函数构造

按照经验来讲，reward 函数采用极其简单的形式，即在执行一次动作后，存活则得一分，死亡则得 -1000 分即可，但是受到游戏编写时的限制，本项目的游戏帧数高从而导致 action 的次数较多，可能在跳跃之后无法进入下一个 state，因此简单的 reward 收敛速度和效果可能并不是很好，因此我们细化了 reward 函数（尽管这可能导致训练更加困难）

我们在执行了一步动作后,如果小鸟依旧存活，就按照纵向区域划分进行给分.

如何判断小鸟依旧存活?

计算小鸟距离管子的水平、垂直距离,小鸟的垂直距离如果不在两个管道中间则相应地扣分,死亡则 reward=-1000，reward 里的纵向距离分类和 state 里的分类标准这两个地方理论上应保持完全一样，但实际应用时不超过 20%的差距是可以接受的,这些分类对 agent 的 action 影响很大，需要仔细调整参数.

```
def reward(Bird,pipe_sprites):
    #首先判断死没死
    if Bird.is_dead == False:
        #没死就计算水平、垂直距离
        for pipe in pipe_sprites:
            dist_to_pipe_bottom = Bird.rect.top - (pipe.rect.top + 320
            dist_to_pipe_horz = pipe.rect.right - Bird.rect.left
            if dist_to_pipe_horz <= 0:
                continue
            break
        #根据执行了一步动作后，垂直距离的大小来进行打分
        if dist_to_pipe_bottom <= -40:
            r = -5
        elif dist_to_pipe_bottom <= 0:
            r = -2
        elif dist_to_pipe_bottom <= 55:
            r = -1
        elif dist_to_pipe_bottom <= 95:
            r = 0
        elif dist_to_pipe_bottom <= 125:
            r = 1
        elif dist_to_pipe_bottom <= 150:
            r = 0
        elif dist_to_pipe_bottom <= 190:
            r = -1
        elif dist_to_pipe_bottom <= 230:
            r = -2
```

2.2.7 优化与参数调整

在项目初期，由于参数设置不合理，导致小鸟的动作没法达到收敛。我们采取的对策如下：观察小鸟为什么会死亡,然后针对性进行参数调整.比如它是撞在管道上方，还是下方. 如果横着撞死，说明预留的下降时间不够，或者它跳太高太低了，那就修改 agent 开始准备下一个管道的位置，如果竖着撞死，说明上下区域不对.

2.3 遇到的问题以及解决方式

2.3.1 绘图问题

希望人运行时可以绘图观察，在服务器上运行时不绘图，可以用条件编译.python 中的一个内置常量 __debug__ ，该常量默认是 True ，但是在执行 python 脚本时加上 -O 会那么该变量就为 False 。利用这个变量可以完成类似 C/C++的条件编译。

```
if __name__ == "__main__":
    if not __debug__ :
        print("This is a debug test")
    else :
        print("This is a usual test")
..$ python3 Example.py
This is a usual test
..$ python3 -O Example.py
This is a debug test
```

尽管有点麻烦，还是能完成测试. 这样会比 cpp 慢很多，因为不是编译成机器码的.

2.3.2 如何在有限时间内评估

我们认为跑个 1w 次，平均能有 300 分算是良好，能有 600 分优秀，能有 1000 分算是非常优秀.先把学习率改到 0.7，跑个一百多轮才能知道效果好不好. 如果跑前 300 次,没有出现明显越来越糟糕的情况，就可以在服务器准备跑 1 万或者 10 万次了

过程中的截图

2.3.3 接口问题

由于 Qlearning 算法和游戏的编写是由两位组员分别完成，而 Qlearning 算法需要获得游戏的一些参数信息，但是二者的接口部分起初没有做好，导致交互困难。

解决方法：首先在游戏状态参数方面，组员通过绘制 2.2.3 中的草图对游戏的参数有了理性的认识，其次将游戏部分的代码加以封装，形成了函数接口方便算法调用。

2.3.4 游戏运行速度较慢

游戏运行速度较慢，导致训练时迭代速度较慢，需要很长时间才能达到收敛。

解决方法：我们提高了游戏的帧率，并且在训练时隐去了动画效果，从而提高了训练速度。

2.3.5 收敛的位置不正确

项目初期参数设置不合理导致收敛的位置不正确。

解决方案：实际上参数设置不合理是我们对 Qlearning 算法没有完全掌握造成的，在学习了算法的具体内容后，我们根据小鸟在运行过程中的表现进行参数的细调，最终取得了较好的效果。

2.3.6 游戏运行时，会带有拖影效果

解决方法：经调试发现，pygame 图像更新的方法是覆盖式的，如果只更新前景而不更新背景，则会导致前一帧前景未删除，造成拖影效果。而它本身不带删除功能。因此要注意每一帧的图像更新顺序，一般来说先背景再前景。

3 团队合作

3.1 任务分工

林炬乙:

1.对 reward 和 get state 进行调优，对于 get state 函数,进行纵向区域划分调整.修改阈值，多设置区域,可以让阈值更加合理

2. 编写了绘图部分，可以让每次数据测试后提取绘图.

3.编写报告的总体框架，修改完善报告.

蒋理:

1. 完成了 Qlearning 算法基本框架的搭建，包括 Qtable 的建立到决策函数的编写，将 bellman 方程转化为可实现的代码。
2. 完成了对状态 state 以及 reward 函数的基本实现，参数有待调整，但已基本能够实现要求。
3. 完成了训练的优化部分，即将动画效果隐去，提高帧率，从而提高训练速度
4. 完成了 2.2.3 草图的绘制方便组员之间交换信息，提高了团队运作效率
5. 完成了游戏代码与 Qlearning 算法的融合，首先实现了项目的正常运转。

蒋宇龙:

1. 编写游戏

2.参数优化

3.贡献运行代码的主机

3.2 编码规范

①基本要求

1.1 程序结构清晰,简单易懂

1.2 代码精简,直接了当，避免垃圾程序。

1.3 尽量使用标准库函数和公共函数。

1.4 项目结构组织（工程文件目录结构是否良好，代码风格是否良好）

②可读性要求

2.1 保持注释与代码完全一致。

2.2 每个源程序文件,都有文件头说明。

2.3 每个函数,都有函数头说明,说明规格见规范。

2.4 主要变量(结构、联合、类或对象)定义或引用时,注释能反映其含义。

2.5 处理过程的每个阶段都有相关注释说明。

2.6 利用缩进来显示程序的逻辑结构

③变量命名

命名有一定的实际意义

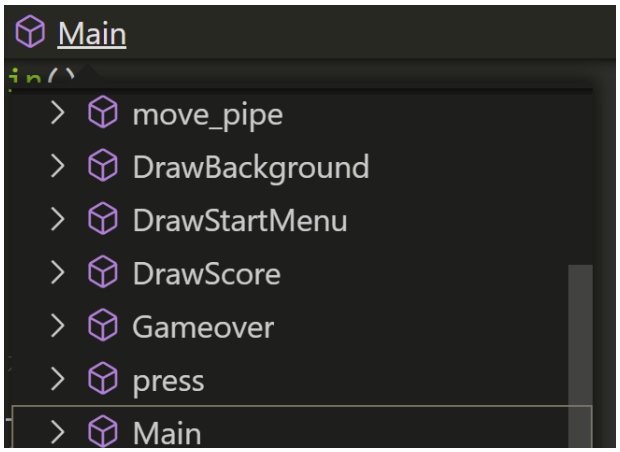
具体例程:

```
if dist_to_pipe_bottom < 0:  # very close or less than 0
    height_category = 0
elif dist_to_pipe_bottom < 40:  # close
    height_category = 1
elif dist_to_pipe_bottom < 75:  # not close
    height_category = 2
elif dist_to_pipe_bottom < 110:  # mid
    height_category = 3
elif dist_to_pipe_bottom < 150:  # far
    height_category = 4
else:
    height_category = 5
```

④函数命名

函数原型说明包括引用外来函数及内部函数,外部引用必须在右侧注明函数来源:模块名及文件名,如是内部函数,只要注释其定义文件名;第一个字母必须使用大写字母,要求用大小写字母组合规范函数命名,必要时可用下划线间隔

示例如下:



3.3 个人收获

林炬乙收获感言:

这是我第一次进行强化学习的训练，了解了强化学习到底是怎么样一个过程，也是看到了强化学习在实际应用中的巨大作用. 感谢队友教会了我许多工程上的具体细节.

蒋理收获感言:

刚开始编写算法部分时，是完全脱离游戏编写的，因为对游戏的参数、接口一无所知，导致编写过程十分煎熬。之后在于组员沟通后才慢慢将游戏与算法融合在了一起。但是由于对算法没有一个十分理性的认识，不知道其具体的工作原理，导致调参十分困难，在完成了项目的基本运转后，花了大量时间用于调参，但是之后回过头看，有很多不必要的调参是可以通过对算法和游戏的学习理解来避免的。总之，这次项目让我对强化学习有了进一步的了解，第一次成功实践了强化学习项目，也从中学到了团队合作的许多知识。

蒋宇龙收获感言:

开始做 project 的时候并没有讲到 Q-learning 和马尔科夫决策过程，以为这种自动学习的 AI 是一个很高

级的过程。不过后来实际做了发现它其实也就是与之前所讲的算法类似的，对于每个状态的评估，并选择最优的策略去下一步，其实际效果如何完全取决于我们对状态评估的准确性，属于传统人工智能的范畴，也没有想象中难。在 project 中接触了 pygame 这个非常方便的编写游戏的库，并通过调试了解了马尔可夫决策的实际迭代过程，根据其迭代结果做优化的过程还是比较有趣的。